coursera | Northeastern University

Foundations of Statistical Learning & Algorithms

Today's goal • 3/3 items

Module 1
Introduction to Linear Algebra and Vector Spaces

Module 2
Linear Transformation and Matrix Decomposition

Module 3
Probability Foundations for Statistical Learning

Module 4
Introduction to Model Evaluation and Optimization

Lesson 1: Model Selection Techniques

- Introduction to Model Selection
  Reading • 5 min
- Bayesian Model Selection
  Reading • 30 min
- Bayesian Model Selection Readings
  Reading • 1h

Lesson 2: Cross Validation

- Introduction to Cross-Validation
  Reading • 25 min
- K-Fold Cross-Validation
  Reading • 2 min
- Leave-One-Out Cross-Validation (LOOCV)
  Reading • 10 min

Lesson 3: Introduction to Optimization Techniques

- Introduction to Optimization Techniques
  Reading • 13 min
- Optimization Using Gradient Descent
  Reading • 40 min
- **Gradient Descent with Momentum**
  Reading • 45 min
- Stochastic Gradient Descent (SGD)
  Reading • 40 min
- Constrained Optimization
  Reading • 16 min
- Lagrange Multipliers
  Reading • 8 min
- Convex Optimization
  Reading • 8 min
- Linear Programming
  Reading • 15 min
- Quadratic Programming
  Reading • 10 min
- Check Your Knowledge: Optimization Techniques
  Graded Assignment • 15 min

Gradient Descent with Momentum

0:12 / 3:02 2x

Gradient Descent With Momentum is an optimization technique designed to improve the convergence speed of standard gradient descent, especially in cases where the optimization landscape presents challenges due to poorly scaled regions. These are areas where the curvature of the function causes the gradient descent steps to oscillate between the "walls" of a valley, leading to slow convergence as the steps become increasingly small and the algorithm approaches the optimum. (Brownlee, 2019).

In many optimization problems, the surface of the objective function can have regions with high curvature, where the gradient varies sharply in one direction but only slightly in another. This creates long, narrow valleys that can trap the gradient descent algorithm, causing it to zigzag between the sides of the valley, taking small steps towards the minimum. This behavior significantly slows down the convergence process.

For a detailed explanation and step-by-step implementation of gradient descent with momentum, refer to this article by Brownlee (2019): Gradient Descent With Momentum From Scratch. (30 minutes)

Introducing Momentum

To address this issue, gradient descent with momentum introduces an additional term that gives the algorithm "memory" of previous updates. This momentum term smooths out the updates by combining the current gradient with a fraction of the previous update. The idea is to allow the optimization process to "remember" the direction it was moving in, thereby reducing oscillations and accelerating convergence through the valleys.

Using the analogy of a heavy ball rolling down a hill, momentum in gradient descent acts like the inertia of the ball, making it less likely to change direction abruptly. As a result, the ball (or the optimization process) continues to move in the same general direction, even when encountering regions of high curvature.

Mathematical Formulation

The update rules in gradient descent with momentum is as follows:

1. Parameter Update

$$x_{i+1} = x_i - \gamma_i (\nabla f\left(x_i\right))^T + \alpha \Delta x_i$$

where:

- x_{i+1} is the updated parameter vector.
- γ_i is the learning rate at iteration i.
- $(\nabla f\left(x_i\right))^T$ is the transpose of the gradient at the current position x_i.
- α is the momentum coefficient, typically between 0 and 1.
- $\Delta x_i = x_i - x_{i-1}$ is the change in the parameter vector from the previous iteration

2. Momentum Update

$$\Delta x_i = \alpha \Delta x_{i-1} - \gamma_{i-1} (\nabla f\left(x_{i-1}\right))^T$$

Here, the update Δx_i is a linear combination of the previous update Δx_{i-1} and the gradient at the previous iteration $\nabla f(x_{i-1})^T$.

The momentum coefficient α controls the influence of the past gradients. A higher value of α gives more weight to the previous updates, leading to smoother and potentially faster convergence.

Go to next item ✓ Completed

Like Dislike Report an issue

Go to next item →